DRIFTLESS — Project Structure & Folder Organization

1. Complete Project Directory Tree

```
c:\Users\AADI\Desktop\driftless\
├── .gitignore
├── CHANGELOG.md
├── DEPLOY.md
├── README.md
├── driftless-extension.zip
│
├── dashboard/                          # Django Web Application Root
│   ├── .env                            # Local environment secrets and flags
│   ├── .env.example                    # Template environment configuration
│   ├── Procfile                        # Process configuration for PaaS
(Heroku/Render)
│   ├── db.sqlite3                      # Local SQLite database file
│   ├── manage.py                       # Django CLI management entrypoint
│   ├── render.yaml                     # Infrastructure-as-code spec for Render.com
│   ├── requirements.txt                # Python dependency manifest
│   │
│   ├── driftless/                      # Django Project Core Configuration Package
│   │   ├── __init__.py
│   │   ├── settings.py                 # Global Django settings, middlewares, apps,
CORS
│   │   ├── urls.py                     # Top-level URL routing mapping to core.urls
│   │   └── wsgi.py                     # WSGI web server entrypoint for production
│   │
│   ├── core/                           # Primary Django Application Module
│   │   ├── __init__.py
│   │   ├── admin.py                    # Django Admin model registrations & filters
│   │   ├── apps.py                     # Core application configuration
│   │   ├── models.py                   # Data models, domain classifier, schema
definitions
│   │   ├── serializers.py              # DRF serializers, payload validators, persona
defaults
│   │   ├── tests.py                    # Automated unit & integration test suite
│   │   ├── urls.py                     # REST API endpoints & HTML template routes
│   │   ├── views.py                    # API view functions, throttling, analytics
logic
│   │   │
│   │   ├── management/                 # Custom Django Management Commands
│   │   │   └── commands/
│   │   │       ├── __init__.py
│   │   │       └── seed_demo.py        # Generates 14 days of realistic demo history
│   │   │
│   │   ├── migrations/                 # Database Schema Migration History
│   │   │   ├── 0001_initial.py
│   │   │   ├── 0002_usagelog_category_usersettings.py
│   │   │   ├── 0003_userprofile.py
│   │   │   ├── 0004_alter_userprofile_persona.py
│   │   │   └── 0005_usersettings_allowed_sites_and_more.py
```

```
│   │   │
│   │   ├── static/                          # Static Frontend Assets
│   │   │   ├── css/
│   │   │   │   └── theme.css                # Shared CSS tokens (Calm Sky & Opal Noir)
│   │   │   └── img/
│   │   │       └── logo.jpeg                # Driftless visual brand identity logo
│   │   │
│   │   └── templates/                       # Server-Rendered HTML5 Interfaces
│   │       ├── auth.html                    # Login & registration portal with persona
picker
│   │       └── dashboard.html               # Analytics dashboard with Chart.js
visualization
│   │
│   └── staticfiles/                         # WhiteNoise static production collection
directory
│
├── extension/                               # Chrome Browser Extension (Manifest V3)
│   ├── manifest.json                        # Extension manifest v3 metadata & permissions
│   ├── background.js                        # Background Service Worker (dwell tracking,
sync, DNR)
│   ├── popup.html                           # Action popup UI with quick stats & focus
controls
│   ├── popup.js                             # Popup controller script & bidirectional sync
trigger
│   ├── blocked.html                         # Intervention splash screen when limits are
breached
│   ├── theme.css                            # Extension-scoped CSS variable design tokens
│   └── icons/                               # Extension action & store icon assets
│       ├── icon-16.png
│       ├── icon-48.png
│       └── icon-128.png
│
├── scripts/                                 # Utility & Build Tooling
│   ├── build_extension.py                   # Python script bundling extension into zip
distribution
│   └── build_extension.sh                   # Shell script alternative for Unix/macOS build
│
└── docs/                                    # Full Codebase Technical Documentation
    ├── PROJECT_OVERVIEW.md
    ├── PROJECT_STRUCTURE.md
    ├── TECH_STACK.md
    ├── FILE_BY_FILE_ANALYSIS.md
    ├── FRONTEND_ARCHITECTURE.md
    ├── CHROME_EXTENSION_ARCHITECTURE.md
    ├── DJANGO_BACKEND.md
    ├── API_REFERENCE.md
    ├── DATABASE_AND_DATA_FLOW.md
    ├── SYSTEM_ARCHITECTURE.md
    ├── FEATURE_CODE_MAPPING.md
    ├── CONFIGURATION_AND_DEPENDENCIES.md
    ├── SECURITY_AND_LIMITATIONS.md
    └── VERIFICATION_REPORT.md
```

2. Directory Responsibilities & Architectural Layering

2.1. Root Level (`/`)

- **Architectural Layer:** Project Configuration & Orchestration
- **Purpose:** Houses cross-cutting documentation (`README.md` , `DEPLOY.md` , `CHANGELOG.md`), version control ignore directives (`.gitignore`), packaged distributables (`driftless-extension.zip`), and top-level subdirectories.
- **Why It Exists:** Provides clean isolation between the browser client (`extension/`), the backend web server (`dashboard/`), build automation (`scripts/`), and technical documentation (`docs/`).

2.2. Extension Layer (`/extension`)

- **Architectural Layer:** Client-Side Browser Extension (Chrome MV3)
- **Purpose:** Executes directly inside the user's browser runtime. Continuously observes navigation events, tracks dwell duration on active browser tabs, enforces limits via dynamic network redirects, displays the quick-action popup, and communicates asynchronously with the Django backend.
- **Key File Types:** JSON (`manifest.json`), JavaScript (`background.js` , `popup.js`), HTML5 (`popup.html` , `blocked.html`), CSS3 (`theme.css`), and PNG icons (`icons/`).
- **Interaction with other layers:**
- Makes periodic REST calls (`POST /api/usage/`) to push uncommitted time deltas.
- Pulls settings and per-site limits from `/api/settings/` and `/api/limits/` .
- Redirects blocked browser navigations locally to `chrome-extension:// ... /blocked.html` .

2.3. Dashboard Backend Layer (`/dashboard`)

- **Architectural Layer:** Server Application & Business Logic
- **Purpose:** Contains the complete Django 4.2 LTS project, including virtual environment references, database persistence, REST API views, URL dispatcher, serializers, and background seed commands.
- **Key Sub-Packages:**
- `driftless/` : Global WSGI, URL routing, and security/settings definitions.
- `core/` : Django app containing models, serializers, views, templates, static files, and database migrations.
- **Interaction with other layers:**
- Ingests data from `extension/` via authenticated Token API requests.
- Serves static assets and pre-rendered HTML5 pages (`auth.html` , `dashboard.html`) to desktop web browsers.
- Interacts with `db.sqlite3` via the Django Object-Relational Mapper (ORM).

2.4. Static Assets Layer (`/dashboard/core/static`)

- **Architectural Layer:** Presentation & Theme Styling
- **Purpose:** Holds compiled CSS stylesheets (`theme.css`) and binary image assets (`logo.jpeg`).
- **Why It Exists:** Decouples styling rules and graphic assets from HTML templates, enabling browser-side caching and static serving through WhiteNoise.
- **Interaction with other layers:**
- Referenced by `dashboard/core/templates/dashboard.html` and `auth.html` via `{% static %}` or `/static/css/theme.css` .

2.5. Templates Layer (`/dashboard/core/templates`)

- **Architectural Layer:** Web Presentation Layer (Server-Rendered HTML)
- **Purpose:** Contains the UI documents rendered by Django's template engine:
- `auth.html` : User onboarding, persona configuration, and credential authentication.
- `dashboard.html` : Analytics visualization, Chart.js graphs, per-site limit configuration modal, category overrides, and data export triggers.
- **Interaction with other layers:**
- Injected with Django context variables (`api_token` , `username` , `persona`) from `views.py` .
- Executes client-side JavaScript that calls REST endpoints using `fetch()` .

2.6. Build Scripts Layer (`/scripts`)

- **Architectural Layer:** Tooling & Deployment Automation
- **Purpose:** Provides automated cross-platform scripts (`build_extension.py` , `build_extension.sh`) to recursively package `extension/` into a compressed release zipfile (`driftless-extension.zip`).
- **Why It Exists:** Enables distribution of the Chrome extension without requiring manual compression utilities or accidental leakage of hidden OS files (`.DS_Store` , `Thumbs.db`).

2.7. Documentation Layer (`/docs`)

- **Architectural Layer:** Knowledge Base & Architectural Reference
- **Purpose:** Stores comprehensive, technical reference documents detailing every facet of DRIFTLESS, including APIs, models, algorithms, security posture, and test evidence.
